

ASSISTED CODING

ASSIGNMENT-4.2

PROGRAM: B.TECH/CSE

SPECIALIZATION : AIML

COURSE TITLE : AI ASSISTANT CODING

SEMESTER : 3RD

NAME OF THE STUDENT: MANDALA.TEJASRI

ENROLLMENT NO: 2403A51171

BATCH NO:01

Task Description#1

- Zero-shot: Prompt AI with only the instruction —
Write a Python function to
generate the Fibonacci sequence up to n terms

PROMPT:

write a code in Python function to generate the Fibonacci sequence up to n term with input is given by user.

Expected Output#1

- A working function without using any sample inputs/outputs

CODE :

```
def generate_fibonacci_sequence(n):
    fibonacci_sequence = []
    a, b = 0, 1
    for _ in range(n):
        fibonacci_sequence.append(a)
        a, b = b, a + b
    return fibonacci_sequence

# Example usage:
try:
    n_terms = int(input("Enter the number of terms for the Fibonacci sequence: "))
    if n_terms < 0:
        print("Please enter a non-negative integer.")
    else:
        fib_sequence = generate_fibonacci_sequence(n_terms)
        print(f"Fibonacci sequence up to {n_terms} terms: {fib_sequence}")
except ValueError:
    print("Invalid input. Please enter an integer.")
```

Enter the number of terms for the Fibonacci sequence: 15
Fibonacci sequence up to 15 terms: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377]

OBSERVATION:

generate_fibonacci_sequence that generates the Fibonacci sequence up to a specified number of terms. It then prompts the user to enter the desired number of terms, handles potential invalid input (non-integer or negative numbers), and prints the resulting Fibonacci sequence

Task Description#2

- **One-shot: Provide one example: Input: 100, Output: 37.78 to help AI generate a function that converts Fahrenheit to Celsius**

Expected Output#2

- **A correct conversion function guided by the single example**

PROMPT:

Generate a function that converts Fahrenheit to Celsius, example:input=100 and output=37.78 with input is given by user.

CODE:

```
def fahrenheit_to_celsius(fahrenheit):
    """Converts Fahrenheit to Celsius.

    Args:
        fahrenheit: The temperature in Fahrenheit.

    Returns:
        The temperature in Celsius.
    """
    celsius = (fahrenheit - 32) * 5/9
    return celsius

# Example usage:
try:
    fahrenheit_temp = float(input("Enter the temperature in Fahrenheit: "))
    celsius_temp = fahrenheit_to_celsius(fahrenheit_temp)
    print(f"{fahrenheit_temp} Fahrenheit is equal to {celsius_temp:.2f} Celsius")
except ValueError:
    print("Invalid input. Please enter a number.")
```

Enter the temperature in Fahrenheit: 100
100.0 Fahrenheit is equal to 37.78 Celsius

OBSERVATION:

This code defines a function to convert Fahrenheit to Celsius and prompts the user for the temperature in Fahrenheit.

Task Description#3

- Few-shot: Give 2–3 examples to create a function

that extracts the domain name from an email address.

Expected Output#3

- Accurate function that returns only the domain portion of an email (e.g., @gmail.com).

PROMPT:

Write a code to extract the domain from mail_id

ex1:input=2403a51350@sru.edu.in
,output=@sru.edu.in

ex2:input=sreeyad4@gmail.com ,output=@gmail.com

CODE:

```
def extract_domain(email_id):  
    try:  
        domain = "@" + email_id.split("@")[1]  
        return domain  
    except IndexError:  
        return "Invalid email format"  
  
# Example usage:  
email1 = "2403a51350@sru.edu.in"  
domain1 = extract_domain(email1)  
print(f"The domain of {email1} is {domain1}")  
  
email2 = "sreeyad4@gmail.com"  
domain2 = extract_domain(email2)  
print(f"The domain of {email2} is {domain2}")  
  
email3 = "invalid_email"  
domain3 = extract_domain(email3)  
print(f"The domain of {email3} is {domain3}")
```

→ The domain of [2403a51350@sru.edu.in](#) is @sru.edu.in
The domain of [sreeyad4@gmail.com](#) is @gmail.com
The domain of invalid_email is Invalid email format

OBSERVATION:

The code in cell defines function `extract_domain` that takes an email address as input. It attempts to split the email address at the "@" symbol and returns the part after the "@" with a "@" prefix to represent the domain. It also includes error handling using a try-except block to catch `IndexError` if the input is not a valid email format (i.e., it doesn't contain an "@").

symbol), in which case it returns "Invalid email format".

Task Description#4

- Compare zero-shot vs few-shot prompting for generating a function that checks whether a word is a palindrome, ignoring punctuation and case.

Expected Output#4

- Output comparison + student explanation on how examples helped the model

ZEROSHOT PROMPT:

Write a function that checks whether a word is a palindrome, ignoring punctuation and case? with user input?

CODE:

```
import re

def is_palindrome(word):

    # Remove punctuation and convert to lowercase
    cleaned_word = re.sub(r'^\w+', '', word).lower()

    # Check if the word is equal to its reverse
    return cleaned_word == cleaned_word[::-1]

# Example usage
word_to_check = input("Enter a word or phrase to check if it's a palindrome: ")
if is_palindrome(word_to_check):
    print(f"{word_to_check} is a palindrome.")
else:
    print(f"{word_to_check} is not a palindrome.")
```

Enter a word or phrase to check if it's a palindrome: RACECAR
'RACECAR' is a palindrome.

FEWSHOT PROMPT:

write a code to check whether input is palindrome or not by user given input EX1: INPUT=RACE,CAR;
OUTPUT=RACECAR EX2:INPUT="MOM'O";
OUTPUT=MOMO

CODE:

```
import re

def is_palindrome_alt(word):

    # Remove punctuation and convert to lowercase
    cleaned_word = re.sub(r'^\w+', '', word).lower()

    left, right = 0, len(cleaned_word) - 1

    while left < right:
        if cleaned_word[left] != cleaned_word[right]:
            return False
        left += 1
        right -= 1

    return True

# Example usage with user input:
word_to_check = input("Enter a word or phrase to check if it's a palindrome (alternative me'
if is_palindrome_alt(word_to_check):
    print(f'{word_to_check}' is a palindrome.")
else:
    print(f'{word_to_check}' is not a palindrome.")
```

Enter a word or phrase to check if it's a palindrome (alternative method): race,car
'race,car' is a palindrome.

OBSERVATION:

"Looking at both palindrome functions again, I see how the examples provided helped clarify the requirements. The examples like 'RACE,CAR' and 'MOM'O' specifically showed that punctuation and case should be ignored. If only a simple example like 'level' was given, the code might not have included the steps to remove punctuation and convert to lowercase. This shows how giving a few varied examples (few-shot) is really useful for making sure the code handles different situations correctly, compared to just one simple example (one-shot)."

Task Description#5

- Use few-shot prompting with 3 sample inputs to generate a function that determines the maximum of three numbers without using the built-in max() function.

Expected Output#5

- A function that handles all cases with correct logic based on example patterns

PROMPT:

generate a python function that determines the maximum of three numbers without using the built-in max() function with user given input? ex1:

a=20,b=30,c=40;output=40

ex2:a=25,b=34,c=50;output=50

ex3:a=765,b=234,c=345;output=765?

CODE:

```
def find_maximum_of_three(a, b, c):
    maximum = a
    if b > maximum:
        maximum = b
    if c > maximum:
        maximum = c
    return maximum

# Example usage with user input:
try:
    num1 = float(input("Enter the first number: "))
    num2 = float(input("Enter the second number: "))
    num3 = float(input("Enter the third number: "))

    maximum_num = find_maximum_of_three(num1, num2, num3)
    print(f"The maximum of {num1}, {num2}, and {num3} is: {maximum_num}")
except ValueError:
    print("Invalid input. Please enter valid numbers.")
```

Enter the first number: 234
Enter the second number: 345
Enter the third number: 567
The maximum of 234.0, 345.0, and 567.0 is: 567.0

OBSERVATION:

The code in this cell defines a function `find_maximum_of_three` that takes three numbers as input. It initializes a variable `maximum` with the value of the first number. Then, it compares the second number with the current maximum and updates maximum if the second number is greater. Finally, it compares the third number with the current maximum and updates it again if the third number is greater. The function returns the final maximum value. The code also includes a part that prompts the user for three

numbers and calls this function to find and print the maximum, with error handling for non-numeric input.